

Template-Guided Specification Mining for GR(1)

Sam Kouteili
Yale University
sam.kouteili@yale.edu

Abstract—We present GR1MINE, a SAT-based tool for mining GR(1) specifications from lasso-shaped traces. GR(1) formulas—of the form $(\varphi_I^e \wedge \Box \varphi_S^e \wedge \bigwedge_i \Box \Diamond J_i) \rightarrow (\varphi_I^g \wedge \Box \varphi_S^g \wedge \bigwedge_j \Box \Diamond G_j)$ —are the fragment of LTL for which reactive synthesis is polynomial, yet no prior tool mines them directly from traces. Existing LTL miners search over both temporal and propositional operators, treating GR(1) as just another LTL formula. We exploit the fixed temporal skeleton of GR(1): the temporal structure is given by the template, so only propositional sub-formulas need to be synthesised. This eliminates the temporal operator semantics—the dominant cost in general LTL mining—and decomposes a single large SAT instance into several small, independent ones. An environment–system variable partition ensures that mined formulas respect the assume–guarantee boundary and are directly suitable for synthesis. On 60 boolean GR(1) specifications from the Spectra project, covering AMBA bus arbiters, generalized buffers, and parameterised arbiters with 16–218 variables, GR1MINE solves 58/60 benchmarks within a 10-minute timeout, scaling to 218 variables. Compared to ATLAS, a state-of-the-art constrained LTL learner, in both unconstrained (ATLAS[LTL]) and GR(1)-constrained (ATLAS[GR1]) configurations, GR1MINE solves 58/60 while ATLAS[LTL] solves 37/60 and ATLAS[GR1] only 25/60. Beyond 76 variables, neither ATLAS configuration finds any solution. Moreover, the GR(1) template guarantees that every mined formula is directly usable for reactive synthesis, whereas ATLAS[LTL] produces predominantly trivial formulas.

I. INTRODUCTION

Generalized Reactivity of Rank 1 (GR(1)) is a fragment of linear temporal logic (LTL) that naturally captures a broad class of reactive systems in which a system must maintain ongoing obligations provided the environment satisfies its own fairness conditions [1]. Several real-world systems, from hardware bus protocols [2] to robotic motion planning [3] and autonomous controllers [4], are expressed as GR(1) specifications. GR(1) is particularly attractive because of its tractability for reactive synthesis: while synthesis from general LTL specifications is doubly exponential in the length of the formula [5], GR(1) synthesis is polynomial in the size of the state space [1]. Tools such as Slugs [6] and Spectra [4] efficiently synthesize correct-by-construction controllers from GR(1) specifications. The AMBA AHB bus arbiter, a widely cited example of synthesis applied to an industrial hardware protocol, was specified and synthesized as a GR(1) problem [2].

However, writing correct GR(1) specifications remains a manual and error-prone process. Moreover, in many practical settings, a formal model may not be available. Consider a blackbox hardware controller or a legacy robotic system for which only execution traces are available. If we can obtain positive traces, representing correct system behaviour,

and negative traces, representing undesirable behaviour (for instance, from test logs with known-good and known-bad executions, or from traces flagged by runtime monitors), we can mine a GR(1) specification consistent with the observed behaviour. Such a mined specification not only reveals the underlying safety and liveness structure of the system, but can be fed directly into existing GR(1) synthesis tools to produce a correct-by-construction controller. This enables a pipeline from observed behaviour to formally verified implementation, with an explicit model of the system.

Existing specification mining tools [7], [8], [9] learn general LTL formulas from positive and negative example traces. However, these tools mine monolithic LTL formulas without any distinction between environment assumptions and system guarantees. Without this decomposition, any subsequent synthesis step must treat all inputs as adversarial, frequently rendering the mined specification unrealisable even when a realisable GR(1) formulation of the same behaviour exists. Recently, ATLAS [10] generalises LTL learning by allowing users to specify structural constraints on the formula to be learned, reducing the problem to MaxSAT via an encoding in first-order relational logic. One can encode the syntactic shape of a GR(1) specification as an ATLAS constraint; however, this approach treats GR(1) structure as a filter on a general LTL search space rather than exploiting it algorithmically. The underlying MaxSAT encoding retains the full generality of LTL semantics over lasso traces, and scalability prohibitively degrades as formula size and the number of traces grow.

In this paper, we present GR1MINE, a SAT-based algorithm that mines GR(1) specifications directly from lasso-shaped traces. GR1MINE fixes the temporal skeleton of the GR(1) template—the $\Box$, $\Box\Diamond$, and initial-condition wrappers are given—and encodes only the unknown propositional content of each component as an independent syntax DAG in a single SAT instance, with hardcoded temporal semantics linking DAG evaluations to lasso positions and consistency constraints enforcing the assume–guarantee classification of traces. An environment–system variable partition restricts which propositions may appear in each component, ensuring that every mined formula respects the assume–guarantee boundary and is directly suitable for synthesis. GR1MINE further exploits the structure of the search space via incremental SAT solving: component DAG constraints are encoded once and shared across template configurations using push/pop, so that only lightweight consistency constraints are added and retracted as the algorithm enumerates over the number of justice and guarantee conditions.

We evaluate GR1MINE on 60 boolean GR(1) specifications from the Spectra benchmark suite [4], covering AMBA bus arbiters, generalized buffers, and parameterised arbiters with 16–218 variables. Compared to ATLAS in both unconstrained (ATLAS[LTL]) and GR(1)-constrained (ATLAS[GR1]) configurations, GR1MINE solves all benchmarks up to 34 variables within a 10-minute timeout, while ATLAS[GR1] solves only 8/22 and ATLAS[LTL] 13/22 on the ≤ 28 -variable subset. The formulas mined by ATLAS[LTL] are predominantly trivial ($\Diamond x_i$, tautologies), yielding 1-state controllers; GR1MINE produces structured GR(1) specifications whose synthesised controllers require 6–73 BDD nodes.

Section II reviews GR(1) and lasso traces. Section III presents the SAT encoding, template enumeration, and incremental solving. Section IV evaluates GR1MINE against ATLAS on the Spectra benchmarks. Section V discusses related work, and Section VI concludes.

II. PRELIMINARIES

A. Linear Temporal Logic

We consider LTL over a set of atomic propositions $AP = \{x_0, \dots, x_{n-1}\}$. Formulas are built from the grammar

$$\varphi ::= x \mid \neg\varphi \mid \varphi \wedge \varphi \mid \bigcirc\varphi \mid \varphi \mathcal{U} \varphi$$

where $x \in AP$, and the derived operators $\Diamond\varphi \equiv \text{true} \mathcal{U} \varphi$ and $\Box\varphi \equiv \neg\Diamond\neg\varphi$. Semantics are defined over infinite words $w \in (2^{AP})^\omega$ in the standard way [11].

B. GR(1) Specifications

A *Generalised Reactivity(1)* (GR(1)) specification [1] over AP has the form

$$\underbrace{(\varphi_I^e \wedge \Box\varphi_S^e \wedge \bigwedge_{i=1}^m \Box\Diamond J_i)}_{\text{assumptions } \mathcal{A}} \rightarrow \underbrace{(\varphi_I^s \wedge \Box\varphi_S^s \wedge \bigwedge_{j=1}^k \Box\Diamond G_j)}_{\text{guarantees } \mathcal{G}} \quad (1)$$

where φ_I^e, φ_I^s are propositional *initial conditions*, φ_S^e, φ_S^s are propositional *safety conditions*, and $J_1, \dots, J_m$ and $G_1, \dots, G_k$ are propositional *justice* (liveness) conditions. Every component is a propositional formula over AP (i.e., built from $\wedge, \vee, \neg$ and variables only); the temporal operators $\Box, \Diamond$ appear only in the fixed positions shown above.

GR(1) is the largest fragment of LTL for which reactive synthesis is known to be polynomial: the three-nested fixed-point algorithm of Piterman et al. [1] runs in $O(n^3mk)$ on a game graph with n states, compared to the 2EXPTIME-complete general LTL synthesis problem [5].

a) Environment–system partition. In reactive synthesis, the propositions are partitioned into *environment* (input) variables AP_e and *system* (output) variables AP_s . Assumptions constrain the environment; guarantees constrain the system. A GR(1) specification is *realisable* only if this contract is respected: justice conditions J_i use only AP_e , guarantee conditions G_j use only AP_s , initial conditions are split (φ_I^e over AP_e , φ_I^s over AP_s), and safety conditions may reference all current-state variables but restrict *next-state* (primed) variables to their respective side— φ_S^e primes only AP_e , φ_S^s primes only AP_s .

C. Lasso Traces

An *ultimately periodic word* (lasso trace) is a pair (u, v) with $u \in (2^{AP})^*$ and $v \in (2^{AP})^+$, representing the infinite word $u \cdot v^\omega$. We store a lasso trace as a finite vector $\sigma = \sigma[0] \dots \sigma[L-1]$ together with a *lasso start* index ℓ , so that $\sigma[\ell..L-1]$ is the repeating loop. For any position $t < L$, the successor is $t+1$ if $t < L-1$, and ℓ otherwise. We write $\text{loop}(\sigma) = \{\ell, \dots, L-1\}$ for the set of loop positions.

A lasso trace σ satisfies $\Box\Diamond J$ iff J holds at some position in $\text{loop}(\sigma)$; it satisfies $\Box\varphi_S$ iff φ_S holds at every position; and it satisfies φ_I iff φ_I holds at position 0. The full formula (1) is an implication: σ satisfies it iff either some assumption fails, or all guarantees hold.

D. The Specification Mining Problem

Definition 1 (Trace separation). *Given a set of positive traces P and negative traces N , the specification mining problem is to find a formula φ such that $\sigma \models \varphi$ for all $\sigma \in P$ and $\sigma \not\models \varphi$ for all $\sigma \in N$. We seek a smallest such formula in a given fragment.*

When the fragment is unrestricted LTL, this is the problem solved by Neider and Gavran [7]. In this work we restrict the fragment to GR(1) and exploit its fixed temporal skeleton.

III. ALGORITHM

We now present GR1MINE, a SAT-based algorithm for mining GR(1) specifications from lasso traces. The central observation is that the temporal structure of a GR(1) formula is *fixed*: every component sits inside a known temporal wrapper ($\Box, \Box\Diamond$, or bare for initial conditions). Only the propositional content of each component—built from $\wedge, \vee, \neg$, and atomic propositions—needs to be discovered. We explain the algorithm constructively, using a running example.

a) Running example. Suppose $AP_e = \{x_0, x_1\}$ and $AP_s = \{x_2, x_3\}$. We are given two positive and two negative lasso traces and wish to find the smallest GR(1) formula separating them.

A. Overview

Figure 1 illustrates GR1MINE’s search strategy. The algorithm performs a two-dimensional search: the outer loop increases the propositional formula depth D (vertical axis), controlling how complex each component can be ($D=1$: a single variable; $D=2$: one operator, e.g. $\neg x_0$; $D=3$: trees, e.g. $x_0 \wedge x_1$); the inner loop enumerates template configurations (m, k, F) (horizontal axis), varying the number of justice and guarantee conditions. For each combination, GR1MINE encodes the mining problem as a SAT instance and checks satisfiability, returning the first solution found.

The GR(1) template (1) is parameterised by the number of justice conditions m , guarantees k , and a subset $F \subseteq \{\text{init}, \text{safe}\}$ indicating which optional component types to include: $\text{init} \in F$ means the template includes initial-condition components (init_{env} on the assumption side, init_{sys} on the guarantee side), and $\text{safe} \in F$ means safety components are included. Since the correct template is unknown, GR1MINE

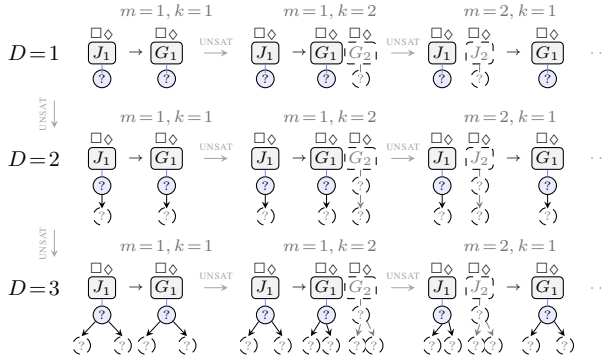


Fig. 1. Two-dimensional search in GR1MINE. **Horizontal:** template enumeration over (m, k) via push/pop. **Vertical:** propositional depth D . $D=1$: single leaves. $D=2$: chains (one operator). $D=3$: trees ($\neg x_0 \wedge x_1$, etc.). Solid = current; dashed = newly added.

searches over configurations $(m, k, F) \in \{1, \dots, M\} \times \{1, \dots, K\} \times 2^{\{init, safe\}}$ in non-decreasing order of total component count $m + k + 2|F|$, trying simpler templates first.

We pre-allocate a set of *component identifiers* $\mathcal{C} = \{J_1, \dots, J_M, G_1, \dots, G_K, init_{env}, init_{sys}, safe_{env}, safe_{sys}\}$, one for each slot in the maximal template. Each $c \in \mathcal{C}$ names an *unknown* propositional formula to be determined by the SAT solver—for instance, $init_{env}$ names the environment initial condition whose content φ_I^e is unknown. The SAT variables x^c, l^c, r^c encode the syntax DAG for c (Section III-B), y^c evaluates it on traces (Section III-C), and h^c captures whether it holds under its temporal wrapper (Section III-D). The ENCODEALL subroutine in Algorithm 1 bundles these steps for all components at a given depth.

The key observation enabling incremental solving is that all of these constraints are *independent* of the template configuration—only the consistency constraints ((6)–(7)) change. For each depth D , we encode all $c \in \mathcal{C}$ once, then iterate over template configurations. Each configuration partitions the active components into assumption-side ($J_i, init_{env}, safe_{env}$) and guarantee-side ($G_j, init_{sys}, safe_{sys}$), asserts the consistency constraints (6)–(7) inside a PUSH/POP scope, and queries the solver. PUSH saves a checkpoint on the solver’s constraint stack; POP retracts all constraints added since the checkpoint, restoring the solver to its prior state. Learned clauses persist across checkpoints, so information gained from one UNSAT configuration prunes subsequent configurations at the same depth.

a) *Example.*: At $D = 1$, the solver calls ENCODEALL for $\mathcal{C}$, creating the SAT variables and constraints on all four traces. It then tries template $(m=1, k=1, F=\emptyset)$: PUSH, assert consistency for $\Box\Diamond J_1 \rightarrow \Box\Diamond G_1$, call SAT. If UNSAT, POP and try $(1, 2, \emptyset)$: $\Box\Diamond J_1 \rightarrow \Box\Diamond G_1 \wedge \Box\Diamond G_2$. If still UNSAT, continue through all configurations at $D = 1$, then increment to $D = 2$, encode larger DAGs (allowing $\neg, \wedge, \vee$), and repeat. Algorithm 1 gives the complete procedure; the remaining subsections detail the four families of constraints added by ENCODEALL: DAG structure (Section III-B), propositional semantics (Section III-C), temporal holds (Section III-D), and

Algorithm 1: GR1MINE

Input : Traces P (pos.), N (neg.), partition (AP_{env}, AP_{sys}) , bounds M, K

Output: Smallest GR(1) formula separating P from N

```

1 Procedure EncodeAll( $\mathcal{S}, \mathcal{C}, D$ )
2   foreach  $c \in \mathcal{C}$  do
3     Init  $x_{i,\lambda}^c, l_{i,j}^c, r_{i,j}^c \forall i < D, \lambda \in \{\wedge, \vee, \neg\} \cup Vars(c)$ 
4     Init  $y_{i,\sigma,t}^c \forall \sigma \in P \cup N, t < |\sigma|$ 
5     Init  $h_{\sigma}^c \forall \sigma \in P \cup N$ 
6      $\mathcal{S}.add(\text{one-hot labels, arity, acyclicity for } x^c, l^c, r^c)$ 
7     // Sec. III-B
8      $\mathcal{S}.add(\text{propositional semantics of } y^c \text{ via } x^c, l^c, r^c)$ 
9     // Sec. III-C
10     $\mathcal{S}.add(h_{\sigma}^c \text{ via Eqs. (2)–(5)})$  // Sec. III-D
11  end procedure
12  $\mathcal{S} \leftarrow \text{SATSOLVER.NEW}()$ 
13  $\mathcal{C} \leftarrow \{J_1, \dots, J_M, G_1, \dots, G_K, init_{env}, init_{sys}, safe_{env}, safe_{sys}\}$ 
14 for  $D = 1, 2, 3, \dots$  do
15   EncodeAll( $\mathcal{S}, \mathcal{C}, D$ ) // permanent constraints
16   for  $(m, k, F) \in \{1, \dots, M\} \times \{1, \dots, K\} \times 2^{\{init, safe\}}$  do
17      $\mathcal{A} \leftarrow \{J_1, \dots, J_m\}$ ;
18      $\mathcal{G} \leftarrow \{G_1, \dots, G_k\}$ 
19     if  $init \in F$  then  $\mathcal{A} \leftarrow \mathcal{A} \cup \{init_{env}\}$ ;  $\mathcal{G} \leftarrow \mathcal{G} \cup \{init_{sys}\}$ 
20     if  $safe \in F$  then  $\mathcal{A} \leftarrow \mathcal{A} \cup \{safe_{env}\}$ ;  $\mathcal{G} \leftarrow \mathcal{G} \cup \{safe_{sys}\}$ 
21      $\mathcal{S}.Push()$  // ckpt: below is retractable
22     foreach  $\sigma \in P$  do
23        $\mathcal{S}.add(\bigwedge_{a \in \mathcal{A}} h_{\sigma}^a \Rightarrow \bigwedge_{g \in \mathcal{G}} h_{\sigma}^g)$ 
24     foreach  $\sigma \in N$  do
25        $\mathcal{S}.add(\bigwedge_{a \in \mathcal{A}} h_{\sigma}^a \wedge \neg(\bigwedge_{g \in \mathcal{G}} h_{\sigma}^g))$ 
26     if  $\mathcal{S}.Sat()$  then return Reconstruct( $model, \mathcal{A}, \mathcal{G}$ )
27      $\mathcal{S}.Pop()$  // retract consistency constraints

```

trace consistency (Section III-E).

B. Component DAGs

Each component c of the GR(1) template is an unknown propositional formula. We represent it as a syntax DAG of D nodes $0, 1, \dots, D-1$, where node $D-1$ is the root and node 0 is always a leaf (atomic proposition). The DAG is described by three families of SAT variables:

- $x_{i,\lambda}^c$: “node i is labelled λ ,” where $\lambda \in \{\wedge, \vee, \neg\} \cup Vars(c)$ and $Vars(c)$ is the set of variables permitted in component c by the environment–system partition (Section II-B).
- $l_{i,j}^c, r_{i,j}^c$: “the left (right) child of node i is node j ,” for $1 \leq i < D, 0 \leq j < i$.
- $y_{i,\sigma,t}^c$: “node i evaluates to true at position t of trace σ .”

Standard constraints enforce exactly one label per node, correct arity ($\wedge/\vee$: two children; $\neg$: one; leaves: none), and acyclicity ($j < i$).

a) *Example at $D = 1$.*: At depth $D = 1$, each component is a single leaf node—just an atomic proposition. For a justice component J_1 , the partition restricts the leaf to environment variables, so the SAT solver chooses among $x_{0,x_0}^{J_1}$ and $x_{0,x_1}^{J_1}$ (exactly one must be true). For a guarantee component G_1 , only system variables are permitted: $x_{0,x_2}^{G_1}$ or $x_{0,x_3}^{G_1}$.

C. Propositional Semantics

The y variables link each DAG to the traces. For a leaf labelled x_p : $y_{i,\sigma,t}^c \Leftrightarrow \sigma[t][p]$. For primed leaves x'_p in safety

components: $y_{i,\sigma,t}^c \Leftrightarrow \sigma[next(t)][p]$. For internal nodes, the semantics follow the operator:

$$\begin{aligned} x_{i,\wedge}^c \wedge l_{i,a}^c \wedge r_{i,b}^c &\Rightarrow (y_{i,\sigma,t}^c \Leftrightarrow y_{a,\sigma,t}^c \wedge y_{b,\sigma,t}^c) \\ x_{i,\vee}^c \wedge l_{i,a}^c \wedge r_{i,b}^c &\Rightarrow (y_{i,\sigma,t}^c \Leftrightarrow y_{a,\sigma,t}^c \vee y_{b,\sigma,t}^c) \\ x_{i,\neg}^c \wedge l_{i,a}^c &\Rightarrow (y_{i,\sigma,t}^c \Leftrightarrow \neg y_{a,\sigma,t}^c) \end{aligned}$$

These are asserted for every trace σ and position t .

a) *Example.*: Suppose the solver assigns $x_{0,x_0}^{J_1} = true$ (i.e., J_1 is the proposition x_0). Then for each trace σ and position t , $y_{0,\sigma,t}^{J_1}$ is forced to equal $\sigma[t][0]$ —the value of x_0 at that position. No search over temporal operators occurs; the y variables are purely propositional.

D. Temporal Holds

Rather than searching over temporal operators, we *hardcode* the temporal semantics of each component’s wrapper. A single Boolean variable h_σ^c captures whether component c holds on trace σ , defined in terms of the root node’s y values. For a trace σ of length L with lasso start ℓ :

$$h_\sigma^{init} \Leftrightarrow y_{D-1,\sigma,0}^{init} \quad (2)$$

$$h_\sigma^{safety} \Leftrightarrow \bigwedge_{t=0}^{L-1} y_{D-1,\sigma,t}^{safety} \quad (3)$$

$$h_\sigma^{J_i} \Leftrightarrow \bigvee_{t \in loop(\sigma)} y_{D-1,\sigma,t}^{J_i} \quad (4)$$

$$h_\sigma^{G_j} \Leftrightarrow \bigvee_{t \in loop(\sigma)} y_{D-1,\sigma,t}^{G_j} \quad (5)$$

a) *Example.*: Continuing with $J_1 = x_0$ and a trace σ whose loop positions are $\{2,3\}$: $h_\sigma^{J_1}$ is true iff $\sigma[2][0] \vee \sigma[3][0]$ —i.e., x_0 is true at some loop position. This directly encodes $\Box\Diamond(x_0)$ without ever introducing $\Box$ or $\Diamond$ as SAT-searchable operators.

E. Consistency Constraints

The consistency constraints tie all components together by enforcing the assume–guarantee semantics of GR(1) on the given traces. Let $\mathcal{A}$ and $\mathcal{G}$ be the sets of active assumption and guarantee components for a template configuration. For each positive trace $\sigma \in P$:

$$\bigwedge_{a \in \mathcal{A}} h_\sigma^a \Rightarrow \bigwedge_{g \in \mathcal{G}} h_\sigma^g \quad (6)$$

For each negative trace $\sigma \in N$:

$$\bigwedge_{a \in \mathcal{A}} h_\sigma^a \wedge \neg(\bigwedge_{g \in \mathcal{G}} h_\sigma^g) \quad (7)$$

a) *Example.*: For the template $\Box\Diamond J_1 \rightarrow \Box\Diamond G_1$ with $\mathcal{A} = \{J_1\}$ and $\mathcal{G} = \{G_1\}$: each positive trace requires $h_{\sigma_+}^{J_1} \Rightarrow h_{\sigma_+}^{G_1}$ (if the assumption holds, the guarantee must hold), and each negative trace requires $h_{\sigma_-}^{J_1} \wedge \neg h_{\sigma_-}^{G_1}$ (the assumption holds *and* the guarantee fails). If the SAT solver finds a satisfying assignment, we read off $x_{0,x_0}^{J_1} = true$ and $x_{0,x_2}^{G_1} = true$, yielding the formula $\Box\Diamond(x_0) \rightarrow \Box\Diamond(x_2)$.

IV. EVALUATION

We evaluate GR1MINE on a suite of GR(1) specifications drawn from real reactive-synthesis benchmarks, comparing against the state-of-the-art constrained LTL learner ATLAS [10] in two configurations. All experiments run on a single core of an Apple M-series machine with a 10-minute timeout per mining task.

A. Benchmarks

We draw our benchmarks from the Spectra project [4], a curated repository of GR(1) specifications from the reactive-synthesis literature. The full repository contains over 660 .spectra files across 10 subdirectories; however, most specifications use syntactic extensions—enumerated types, integer variables, array indexing, parameterized patterns, and macro definitions—that require a booleanisation step beyond the scope of this work. We restrict attention to the *boolean-only* subsets in the bloemDebugging [12] and cimattiAnalyzing [13] directories, which yield **60 parseable specifications** spanning 16–218 variables. The benchmarks represent three families of hardware-protocol specifications:

- **AMBA AHB bus arbiter** (15 specs, 16–40 vars): models the ARM AHB arbitration protocol with request, grant, lock, and ready signals for 1–5 bus masters. Variants include guaranteed fairness, transaction constraints, and weakened assumptions.
- **Generalized buffer** (12 specs, 24–97 vars): a producer–consumer buffer connecting 5–40 sources to receivers via request–acknowledge handshakes (GenBuf protocol).
- **Parameterised arbiter** (33 specs, 22–218 vars): multi-master arbiters with 2–100 competing requesters (A2, A4, A5, G5, G20, and G100 families), with fairness and safety variants.

For each benchmark, we generate 25 positive and 25 negative lasso traces using a mix of two strategies. Half the traces in each class are produced by a *SAT-based* generator that constructs assumption-satisfying traces via Z3: positive traces satisfy both assumptions and guarantees, while negative traces satisfy the assumptions but violate at least one guarantee. The remaining traces are generated by *random sampling* of lasso traces of length 4–6. This mixed strategy is critical: purely random traces often yield trivial separators that do not capture the reactive assume–guarantee structure, while purely SAT-based traces may overfit to the generator’s construction patterns.

B. Baselines

We compare GR1MINE against ATLAS [10], a recent constrained LTL learner that encodes the mining problem as a MaxSAT instance via the Alloy modelling language and the AlloyMax solver. ATLAS accepts user-specified structural constraints in the form of Alloy predicates, allowing it to restrict the search space. We evaluate two configurations:

- **ATLAS[LTL]**: unconstrained LTL mining over the full operator set $\{\Box, \Diamond, \bigcirc, \mathcal{U}, \wedge, \vee, \neg, \rightarrow\}$ with no template

TABLE I

RESULTS BY BENCHMARK FAMILY. *Slv*: SOLVED WITHIN 10 MIN. *Time*: AVG. SECONDS (SOLVED ONLY). *|C|*: AVG. CONTROLLER SIZE—BDD NODES (GR1MINE, ATLAS[GR1]) OR MEALY STATES (ATLAS[LTL]).

Family	$ AP $	n	GR1MINE				ATLAS[LTL]				ATLAS[GR1]			
			Slv	Time	R	$ C $	Slv	Time	R	$ C $	Slv	Time	R	$ C $
AMBA (1)	16	3	3	174	3	14	0	—	—	—	0	—	—	—
A2+AMBA (2)	22	8	8	265	8	65	0	—	—	—	0	—	—	—
G5+GenBuf (5)	24	8	8	11	8	11	8	7	6	1	5	53	5	10
AMBA (3)	28	3	3	30	3	9	2	12	2	1	1	97	1	14
A4+AMBA (4)	34	8	8	19	8	14	8	19	7	1	7	115	7	13
GenBuf (10)	35	3	3	25	3	9	3	54	2	1	2	76	2	8
A5+AMBA (5)	40	8	8	22	7	11	8	34	7	1	8	89	8	9
G20+GenBuf (20)	56	8	6	50	6	10	6	106	4	1	2	180	2	8
GenBuf (30)	76	3	3	3	3	9	2	372	2	1	0	—	—	—
GenBuf (40)	97	3	3	48	3	15	0	—	—	—	0	—	—	—
G100	218	5	5	86	5	13	0	—	—	—	0	—	—	—
Total		60	58		56		37		30		25		25	

or partition constraints. Mined formulas are synthesised via `ltlstxt` [14] (Mealy machine output).

- **ATLAS[GR1]**: we encode a GR(1) template constraint and the environment–system variable partition as Alloy predicates (see Appendix A), restricting the root to an implication between $\Box\Diamond$ subformulas with appropriate literal restrictions. Mined formulas are synthesised via `slugs` [6].

GR1MINE formulas are also synthesised via `slugs`, as they are natively in GR(1) form.

C. Results

Table I summarises the results grouped by variable count. Detailed per-benchmark results including mined formulas are given in Appendix A.

GR1MINE solves 58 of 60 benchmarks (97%) within the timeout, including all benchmarks up to 218 variables. ATLAS[LTL] solves 37/60 (62%), while ATLAS[GR1] solves only 25/60 (42%). The two GR1MINE timeouts are 56-variable GenBuf specifications that require depth $D \geq 4$. ATLAS[GR1] times out on all 16- and 22-variable benchmarks: encoding the GR(1) template as Alloy predicates atop full LTL semantics proves prohibitively expensive at these sizes. At 34–40 variables, all three tools solve most benchmarks, but GR1MINE and ATLAS[LTL] are faster than ATLAS[GR1] (19–22 s vs. 89–115 s on average). Beyond 56 variables, ATLAS[GR1] drops to 2/8 solved, and at 76+ variables neither ATLAS configuration can find any GR(1) formula, while GR1MINE continues solving up to 218 variables in under 3 minutes.

Figure 2 shows a cactus plot of cumulative benchmarks solved as a function of wall-clock time. GR1MINE is slower than ATLAS[LTL] on the 24-variable benchmarks (G5 and GenBuf-5 families), where ATLAS finds small formulas in 5–10 s compared to GR1MINE’s 10–25 s. However, GR1MINE continues solving through the harder benchmarks while ATLAS[LTL] stalls at 37 solved and ATLAS[GR1] at 25. Figure 3 gives a per-benchmark comparison: most points lie above the diagonal, indicating ATLAS is slower or timed out.

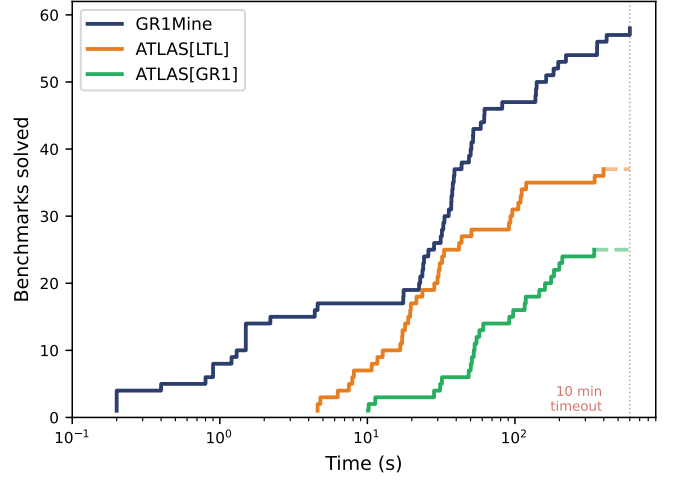


Fig. 2. Cactus plot: cumulative benchmarks solved vs. wall-clock time.

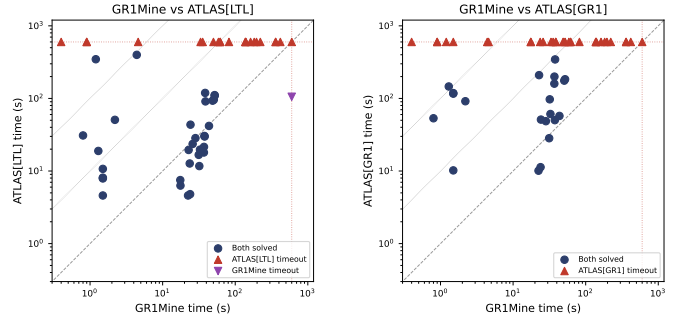


Fig. 3. Scatter plots comparing mining time (log scale). Points above the diagonal indicate GR1MINE is faster. Triangles denote timeouts.

The ATLAS[LTL] formulas are flat LTL expressions outside the GR(1) fragment. Many are *trivial*: $\Diamond x_i$ (“eventually x_i is true”), $x_i \rightarrow x_j$, or simple disjunctions. Several are *tautologies* of the form $\Box(\bigcirc(x \rightarrow x))$ —a formula true on every trace (see Appendix A). None captures the assume–guarantee structure of the underlying specification.

By contrast, GR1MINE produces formulas in canonical GR(1) form with justice conditions, safety invariants, and initial conditions on both the assumption and guarantee sides. On the 25 benchmarks where both GR1MINE and ATLAS[GR1] produce results, the formulas are structurally similar—often agreeing on the justice and guarantee variables—but GR1MINE tends to discover richer templates with multiple justice conditions and safety components, while ATLAS[GR1] is restricted to a single-justice, single-guarantee template imposed by its Alloy constraint (see Appendix A).

Since the purpose of mining GR(1) specifications is to enable reactive synthesis, we evaluate the quality of the mined formulas by synthesising controllers. All 58 GR1MINE formulas are realisable (100%); all 25 ATLAS[GR1] formulas are realisable; but only 30 of 37 ATLAS[LTL] formulas are realisable (81%). The unrealisable ATLAS[LTL] formulas are predominantly trivial ($\Diamond x_i$, $x_i \mathcal{U} x_j$) that happen to be

unsatisfiable by any reactive strategy.

V. RELATED WORK

a) Specification mining from traces.: Flie [7] learns the smallest LTL formula separating positive from negative lasso traces via a SAT encoding over syntax DAGs—our propositional encoding builds on this framework but eliminates temporal operator search by fixing the GR(1) skeleton. In the finite-trace setting, Scarlet [9] learns LTL_f formulas via scalable anytime algorithms, and Bolt [15] reduces LTL_f learning to Boolean set cover. Fijalkow and Lagarde [16] establish complexity bounds for passive LTL learning, showing NP-hardness even for restricted fragments. All of these approaches mine monolithic temporal formulas without distinguishing environment assumptions from system guarantees, making the resulting specifications unsuitable for reactive synthesis without manual restructuring into an assume–guarantee form.

b) Constrained specification mining.: Texada [8] was among the first to introduce user-specified property templates into specification mining, allowing users to constrain the syntactic shape of mined properties. LTL-Sketcher [17] extends this idea to sketch completion: given a partial LTL formula with holes, it fills them from positive and negative examples using a SAT encoding. ATLAS [10] further generalises structural constraints via first-order relational logic in Alloy, reducing constrained LTL learning to MaxSAT. One can encode GR(1) structure as an ATLAS constraint (Appendix A), but the underlying encoding retains full LTL temporal semantics, treating GR(1) as a filter on the general search space rather than exploiting it algorithmically. Orthogonally, work on GR(1) assumption mining [18], [19] automatically generates or refines environment assumptions for a *given* set of system guarantees, while specification debugging [20] localises errors in existing GR(1) specifications. These approaches assume that part of the specification is already known; GR1MINE mines the complete assume–guarantee formula from traces alone. More broadly, GR1MINE differs from all of the above in that the GR(1) template is not a constraint layered atop a general LTL miner but the foundation of the encoding itself: temporal semantics are hardcoded, the search is purely propositional, and incremental solving shares work across template configurations.

VI. CONCLUSION

We presented GR1MINE, a SAT-based algorithm for mining GR(1) specifications from lasso traces. By fixing the temporal skeleton of the GR(1) template and encoding only propositional content, GR1MINE eliminates the need to search over temporal operators and produces formulas that are directly suitable for reactive synthesis. An environment–system variable partition ensures that mined specifications respect the assume–guarantee boundary, and incremental SAT solving enables efficient enumeration over template configurations.

On 60 boolean Spectra benchmarks with 16–218 variables, GR1MINE solves 58/60 within a 10-minute timeout, scaling to 218 variables—well beyond the reach of ATLAS, which solves

at most 37/60 (unconstrained) or 25/60 (GR(1)-constrained) and cannot handle any benchmark beyond 76 variables.

Future work includes extending the trace generator and miner to larger variable counts, incorporating a CEGIS loop that iteratively refines the mined specification against a ground-truth oracle, and supporting richer Spectra specifications through automatic booleanisation of enumerated types and integer variables.

REFERENCES

- [1] N. Piterman, A. Pnueli, and Y. Sa’ar, “Synthesis of reactive(1) designs,” in *Proc. Verification, Model Checking, and Abstract Interpretation (VMCAI)*, 2006, pp. 364–380.
- [2] R. Bloem, S. Galler, B. Jobstmann, N. Piterman, A. Pnueli, and M. Weiglhofer, “Automatic hardware synthesis from specifications: A case study,” in *Proc. Design, Automation and Test in Europe (DATE)*, 2007, pp. 1–6.
- [3] H. Kress-Gazit, G. E. Fainekos, and G. J. Pappas, “Temporal-logic-based reactive mission and motion planning,” *IEEE Transactions on Robotics*, vol. 25, no. 6, pp. 1370–1381, 2009.
- [4] S. Maoz and J. O. Ringert, “GR(1) synthesis for LTL specification patterns,” in *Proc. Joint Meeting on European Software Engineering Conference and Symp. Foundations of Software Engineering (ESEC/FSE)*, 2015, pp. 96–106.
- [5] A. Pnueli and R. Rosner, “On the synthesis of a reactive module,” in *Proc. ACM Symp. Principles of Programming Languages (POPL)*, 1989, pp. 179–190.
- [6] R. Ehlers and V. Raman, “Slugs: Extensible GR(1) synthesis,” in *Proc. Computer Aided Verification (CAV)*, 2016, pp. 333–339.
- [7] D. Neider and I. Gavran, “Learning linear temporal properties,” in *Proc. Formal Methods in Computer-Aided Design (FMCAD)*, 2018, pp. 1–10.
- [8] C. Lemieux, D. Park, and I. Beschastnikh, “General LTL specification mining,” in *Proc. Automated Software Engineering (ASE)*, 2015, pp. 81–92.
- [9] R. Raha, R. Roy, N. Fijalkow, and D. Neider, “Scalable anytime algorithms for learning fragments of linear temporal logic,” in *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2022, pp. 263–280.
- [10] R. Zhang, C. Zhang, R. Meira-Goes, and D. Garlan, “ATLAS: Constrained specification learning from examples with AlloyMax,” in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2025.
- [11] A. Pnueli, “The temporal logic of programs,” in *Proc. IEEE Symp. Foundations of Computer Science (FOCS)*, 1977, pp. 46–57.
- [12] R. Bloem, B. Jobstmann, N. Piterman, A. Pnueli, and Y. Sa’ar, “Synthesis of reactive(1) designs,” vol. 78, no. 3, 2012, pp. 911–938.
- [13] A. Cimatti, L. Geatti, N. Gigante, A. Montanari, and S. Tonetta, “Expressiveness of extended bounded response LTL,” in *Proc. LPAR*, 2020.
- [14] T. Michaud and M. Colange, “Reactive synthesis from LTL specification with spot,” in *Proc. Int. Workshop on Synthesis (SYNT)*, 2018.
- [15] G. Bathie, N. Fijalkow, T. Matricon, B. Mouillon, and P. Vandenhover, “ LTL_f learning meets boolean set cover,” in *Proc. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2026.
- [16] N. Fijalkow and G. Lagarde, “The complexity of learning linear temporal logic,” in *Proc. Int. Conf. Grammatical Inference (ICGI)*, 2021, pp. 237–250.
- [17] S. Lutz, D. Neider, and R. Roy, “Specification sketching for linear temporal logic,” in *Proc. Automated Technology for Verification and Analysis (ATVA)*, 2023, pp. 26–48.
- [18] K. Chatterjee, T. A. Henzinger, and B. Jobstmann, “Environment assumptions for synthesis,” in *Proc. Concurrency Theory (CONCUR)*, 2008, pp. 147–161.
- [19] W. Li, L. Dworkin, and S. A. Seshia, “Mining assumptions for synthesis,” in *Proc. ACM/IEEE Int. Conf. Formal Methods and Models for Codeign (MEMOCODE)*, 2011, pp. 43–50.
- [20] R. Könighofer, G. Hofferek, and R. Bloem, “Debugging formal specifications: A practical approach using model-based diagnosis and counterstrategies,” in *Int. Journal on Software Tools for Technology Transfer (STTT)*, vol. 15, 2013, pp. 563–583.

APPENDIX

Figure 4 shows the Alloy constraint appended to ATLAS trace files to encode the GR(1) template and environment–system variable partition. The constraint restricts the root node to an implication (`ImPLY`) whose left and right children are $\Box\Diamond$ subformulas (encoded as $G \rightarrow F$ chains), with literal restrictions ensuring that assumption literals draw from environment variables and guarantee literals from system variables.

```
fact {
  root in ImPLY
  root.l in G
  root.l.l in F
  root.r in G
  root.r.l in F
  all n: root.l.l.l.l.*(l+r) |
    n in And + Or + Neg + Literal
  all n: root.r.l.l.l.*(l+r) |
    n in And + Or + Neg + Literal
  all n: root.l.l.l.l.*(l+r) & Literal |
    n in <env_literals>
  all n: root.r.l.l.l.*(l+r) & Literal |
    n in <sys_literals>
}
```

Fig. 4. Alloy constraint for ATLAS[GR1]. `<env_literals>` and `<sys_literals>` are replaced with the benchmark’s environment and system variable names (e.g., `x0 + x1 + ... + x4` for 5 environment variables).

Table II presents per-benchmark results for all 60 Spectra benchmarks, including the mined GR1MINE formula. For each tool, we report the mining time (seconds), realizability, and controller size—BDD nodes for GR1MINE and ATLAS[GR1], Mealy states for ATLAS[LTL]. Timeouts are marked TO (10 min).

TABLE II

PER-BENCHMARK EVALUATION RESULTS WITH MINED GRIMINE FORMULAS. $|V|$: TOTAL VARIABLES. TIME: MINING TIME IN SECONDS. R: REALIZABILITY (✓ = REALIZABLE, ✗ = UNREALIZABLE, - = NOT MINED). $|C|$: CONTROLLER SIZE (BDD NODES OR MEALY STATES).

Benchmark	V	GR1[MINE]	ATLAS[LTL]	ATLAS[GR1]	GR1[MINE Formula]			
AMBA AHB bus arbiter (16–40 vars)								
amba_fair_1	16	138 ✓	–	TO –	–	$((\neg x0) \vee (\neg x0) \wedge \Box \Diamond(x2 \vee x0)) \rightarrow (x9 \vee x7 \wedge \Box \Diamond((\neg x7) \wedge (\neg x7)) \wedge \Box \Diamond(x9 \wedge x10) \wedge \Box \Diamond(x11 \wedge x6))$		
amba_trans_1	16	163 ✓	–	TO –	–	$(\Box((\neg \bigcirc(x2)) \vee \bigcirc(x1)) \wedge \Box \Diamond((x3 \vee x0) \vee (x3 \vee x0))) \rightarrow (\Box((\neg x14) \vee ((\neg x14) \wedge x14)) \wedge \Box \Diamond(x10 \wedge (\neg x11)))$		
amba_woaf_1	16	222 ✓	14	TO –	–	$(\neg(x3 \vee x4) \wedge \Box \Diamond((x3 \vee x0) \vee x3)) \rightarrow (x8 \wedge (\neg x14) \wedge \Box \Diamond(x7 \wedge (x7 \wedge x10)))$		
amba_fair_2	22	139 ✓	–	TO –	–	$(\Box((\neg x4) \vee x3) \wedge \Box \Diamond(x1 \vee (\neg x3))) \rightarrow (\Box(\bigcirc(x17) \vee (\neg x20)) \wedge \Box \Diamond((\neg x19) \wedge x10))$		
amba_trans_2	22	360 ✓	–	TO –	–	$(\Box(x1 \vee (\neg x2)) \wedge \Box \Diamond((\neg x4) \vee x6)) \rightarrow (\Box(\bigcirc(x11) \vee (\neg \bigcirc(x20))) \wedge \Box \Diamond((\neg x19) \wedge x12))$		
amba_woaf_2	22	62 ✓	–	TO –	–	$(\neg x4 \wedge \Box \Diamond(x3 \vee x3)) \rightarrow (x9 \vee x9 \wedge \Box \Diamond(\neg x11) \wedge \Box \Diamond(\neg x18) \wedge \Box \Diamond(\neg x19) \wedge \Box \Diamond(\neg x14))$		
amba_fair_3	28	32 ✓	11	12 ✓	1	97 ✓	14	$(\Box(\neg x4) \wedge \Box \Diamond(x0 \wedge x0)) \rightarrow (\Box(\neg \bigcirc(x9)) \wedge \Box \Diamond(x14 \vee x14))$
amba_trans_3	28	33 ✓	8	TO –	–	TO –	–	$(\neg x6 \wedge \Box \Diamond(\neg x4)) \rightarrow (x13 \vee x13 \wedge \Box \Diamond(\neg x20))$
amba_woaf_3	28	24 ✓	8	13 ✓	1	TO –	–	$(\neg x6 \wedge \Box \Diamond(\neg x7)) \rightarrow (x15 \vee x15 \wedge \Box \Diamond(\neg x24))$
amba_fair_4	34	0.2 ✓	8	17 ✓	1	32 ✓	8	$(\Box \Diamond(x0)) \rightarrow (\Box \Diamond(x24))$
amba_trans_4	34	37 ✓	11	18 ✓	1	160 ✓	23	$(\Box(\neg x6) \wedge \Box \Diamond(\neg x7)) \rightarrow (\Box(\neg \bigcirc(x25)) \wedge \Box \Diamond(\neg x30))$
amba_woaf_4	34	26 ✓	11	24 ✗	–	TO –	–	$(\Box(\neg x6) \wedge \Box \Diamond(\neg x5)) \rightarrow (\Box(\bigcirc(x17) \vee \bigcirc(x17)) \wedge \Box \Diamond(x33 \vee x33))$
amba_fair_5	40	0.2 ✓	8	32 ✓	1	53 ✓	8	$(\Box \Diamond(x0)) \rightarrow (\Box \Diamond(x19))$
amba_trans_5	40	38 ✓	22	30 ✓	1	344 ✓	13	$(\Box \Diamond(\neg x0)) \rightarrow (\Box \Diamond(x38 \vee x38) \wedge \Box \Diamond(x18 \wedge x18))$
amba_woaf_5	40	24 ✓	8	44 ✓	1	51 ✓	8	$(\Box \Diamond(\neg x3)) \rightarrow (\Box \Diamond(x39 \wedge x39))$
Parameterised arbiter (22–218 vars)								
A2woef1	22	36 ✓	65	TO –	–	TO –	–	$(\neg x0 \wedge \Box \Diamond(\neg x4)) \rightarrow (x10 \wedge x10 \wedge \Box \Diamond(x9 \vee x9) \wedge \Box \Diamond(\neg x11) \wedge \Box \Diamond(\neg x18))$
A2wsf1	22	140 ✓	–	TO –	–	TO –	–	$(\neg(x3 \vee x3) \wedge \Box \Diamond(x0 \wedge (x0 \vee x0))) \rightarrow (x16 \vee x9 \wedge \Box \Diamond(x8 \wedge x21) \wedge \Box \Diamond((x14 \wedge x14) \wedge x14) \wedge \Box \Diamond((\neg x21) \vee (\neg x21)))$
A2wsf2	22	601 ✓	–	TO –	–	TO –	–	$(\Box(x3 \vee (\neg x4)) \wedge \Box \Diamond((\neg x2) \vee ((\neg x2) \wedge x2))) \rightarrow (\Box(\neg(\bigcirc(x18) \vee x18)) \wedge \Box \Diamond((x21 \vee x21) \vee ((x21 \vee x21) \wedge (x21 \vee x21))))$
A2wst1	22	418 ✓	–	TO –	–	TO –	–	$(\Box((\neg x4) \vee x3) \wedge \Box \Diamond(x2 \vee (\neg x3))) \rightarrow (\Box((\neg x20) \vee x13) \wedge \Box \Diamond(x21 \wedge (\neg x11)))$
A2wst2	22	361 ✓	–	TO –	–	TO –	–	$(\Box(x3 \vee (\neg x4)) \wedge \Box \Diamond((x0 \vee (x0 \wedge x0)) \vee x0)) \rightarrow (\Box((\neg x18) \vee \bigcirc(x14)) \wedge \Box \Diamond(x10 \wedge (\neg x14)))$
G5woef1	24	18 ✓	11	8 ✓	1	TO –	–	$(\Box(\neg x7) \wedge \Box \Diamond(x8 \vee x8)) \rightarrow (\Box(\bigcirc(x16) \vee \bigcirc(x16)) \wedge \Box \Diamond(\neg x22))$
G5wsf1	24	2 ✓	7	8 ✓	1	TO –	–	$(\Box(\bigcirc(x8)) \wedge \Box \Diamond(x8)) \rightarrow (\Box(\bigcirc(x23)) \wedge \Box \Diamond(x23))$
G5wsf2	24	24 ✓	12	5 ✓	1	11 ✓	11	$(\Box \Diamond(\neg x0) \wedge \Box \Diamond(x6 \vee x6)) \rightarrow (\Box \Diamond(x22 \vee x22))$
G5wst1	24	22 ✓	12	5 ✓	1	10 ✓	11	$(\Box \Diamond(\neg x1) \wedge \Box \Diamond(\neg x6)) \rightarrow (\Box \Diamond(x17 \vee x17))$
G5wst2	24	2 ✓	12	8 ✓	1	116 ✓	9	$(\Box(\bigcirc(x8)) \wedge \Box \Diamond(x5)) \rightarrow (\Box(x23) \wedge \Box \Diamond(x18))$
A4woef1	34	23 ✓	27	20 ✓	1	209 ✓	23	$(\Box \Diamond(\neg x10) \wedge \Box \Diamond(x1 \wedge x1)) \rightarrow (\Box \Diamond(x14 \vee x14) \wedge \Box \Diamond(x16 \wedge x16))$
A4wsf1	34	0.2 ✓	8	17 ✓	1	31 ✓	8	$(\Box \Diamond(x0)) \rightarrow (\Box \Diamond(x15))$
A4wsf2	34	37 ✓	15	22 ✓	1	200 ✓	9	$(\Box(\neg x6) \wedge \Box \Diamond(\neg x2)) \rightarrow (\Box(\neg x20) \wedge \Box \Diamond(x33 \vee x33))$
A4wst1	34	1 ✓	26	19 ✓	1	146 ✓	14	$(\Box \Diamond(x3) \wedge \Box \Diamond(x7)) \rightarrow (\Box \Diamond(x14) \wedge \Box \Diamond(x32))$
A4wst2	34	31 ✓	8	17 ✓	1	28 ✓	8	$(\Box \Diamond(\neg x0)) \rightarrow (\Box \Diamond(x19 \vee x19))$
A5woef1	40	28 ✓	12	28 ✓	1	49 ✓	11	$(\Box(\neg x37) \wedge \Box \Diamond(x9 \wedge x9)) \rightarrow (\Box(\neg x5) \wedge \Box \Diamond(x37 \vee x37))$
A5wsf1	40	38 ✓	8	30 ✓	1	50 ✓	8	$(\Box \Diamond(\neg x12)) \rightarrow (\Box \Diamond(x17 \vee x17))$
A5wsf2	40	44 ✗	–	42 ✗	–	57 ✓	11	$(\Box(\neg \bigcirc(x4)) \wedge \Box \Diamond(\neg x2)) \rightarrow (\Box(\neg x12) \wedge \Box \Diamond(x37 \vee x37))$
A5wst1	40	0.8 ✓	13	31 ✓	1	53 ✓	8	$(\Box \Diamond(x9) \wedge \Box \Diamond(x1)) \rightarrow (\Box \Diamond(x39))$
A5wst2	40	0.2 ✓	8	33 ✓	1	55 ✓	8	$(\Box \Diamond(x0)) \rightarrow (\Box \Diamond(x18))$
G20woef1	56	38 ✓	10	119 ✓	1	TO –	–	$(\Box(\neg \bigcirc(x19)) \wedge \Box \Diamond(\neg x11)) \rightarrow (\Box(\bigcirc(x47) \vee \bigcirc(x47)) \wedge \Box \Diamond(\neg x52))$
G20wsf1	56	52 ✓	12	109 ✗	–	TO –	–	$(\Box \Diamond(\neg x7) \wedge \Box \Diamond(\neg x20)) \rightarrow (\Box \Diamond(x50 \wedge x50))$
G20wsf2	56	51 ✓	8	96 ✓	1	176 ✓	8	$(\Box \Diamond(\neg x0)) \rightarrow (\Box \Diamond(x55 \vee x55))$
G20wst1	56	52 ✓	8	111 ✓	1	184 ✓	8	$(\Box \Diamond(\neg x12)) \rightarrow (\Box \Diamond(x46 \vee x46))$
G20wst2	56	59 ✓	12	TO –	–	TO –	–	$(\Box(\neg \bigcirc(x6)) \wedge \Box \Diamond(x23 \vee x23)) \rightarrow (\Box(x47 \wedge x47) \wedge \Box \Diamond(x46 \vee x46))$
G100woef1	218	50 ✓	12	TO –	–	TO –	–	$(x103 \wedge \Box(\bigcirc(x103)) \wedge \Box \Diamond(x101)) \rightarrow (x163 \wedge \Box(x103) \wedge \Box \Diamond(x150))$
G100wsf1	218	0.9 ✓	8	TO –	–	TO –	–	$(\Box \Diamond(x61)) \rightarrow (\Box \Diamond(x217))$
G100wsf2	218	183 ✓	8	TO –	–	TO –	–	$(\Box \Diamond(\neg x34)) \rightarrow (\Box \Diamond(x206 \wedge x206))$
G100wst1	218	0.9 ✓	8	TO –	–	TO –	–	$(\Box \Diamond(x100)) \rightarrow (\Box \Diamond(x107))$
G100wst2	218	196 ✓	27	TO –	–	TO –	–	$(\Box \Diamond(\neg x63) \wedge \Box \Diamond(\neg x12)) \rightarrow (\Box \Diamond(\neg x209) \wedge \Box \Diamond(x206 \vee x206))$
Generalized buffer (24–97 vars)								
gb_fair_5	24	2 ✓	11	11 ✗	–	118 ✓	9	$(\Box(x8) \wedge \Box \Diamond(x2)) \rightarrow (\Box(\bigcirc(x19)) \wedge \Box \Diamond(x18))$
gb_trans_5	24	2 ✓	9	5 ✗	–	10 ✓	11	$(\Box(x8) \wedge \Box \Diamond(x8)) \rightarrow (\Box(x8) \wedge \Box \Diamond(x23))$
gb_woaf_5	24	18 ✓	11	6 ✓	1	TO –	–	$(\Box(\neg x7) \wedge \Box \Diamond(x4 \vee x4)) \rightarrow (\Box(\bigcirc(x10) \wedge \bigcirc(x10)) \wedge \Box \Diamond(x13 \vee x13))$
gb_fair_10	35	39 ✓	11	91 ✗	–	TO –	–	$(\Box(\neg x11) \wedge \Box \Diamond(\neg x8)) \rightarrow (\Box(\neg \bigcirc(x24)) \wedge \Box \Diamond(\neg x25))$
gb_trans_10	35	33 ✓	8	20 ✓	1	61 ✓	8	$(\Box \Diamond(\neg x11)) \rightarrow (\Box \Diamond(x26 \vee x26))$
gb_woaf_10	35	2 ✓	8	51 ✓	1	92 ✓	9	$(x13 \wedge \Box \Diamond(x11)) \rightarrow (x23 \wedge \Box \Diamond(x34))$
gb_fair_20	56	TO –	–	105 ✗	–	TO –	–	–
gb_trans_20	56	49 ✓	8	93 ✓	1	TO –	–	$(\Box \Diamond(\neg x1)) \rightarrow (\Box \Diamond(x50 \vee x50))$
gb_woaf_20	56	TO –	–	TO –	–	TO –	–	–
gb_fair_30	76	5 ✓	7	TO –	–	TO –	–	$(\Box(\bigcirc(x33)) \wedge \Box \Diamond(x6)) \rightarrow (\Box(\bigcirc(x75)) \wedge \Box \Diamond(x75))$
gb_trans_30	76	1 ✓	12	347 ✓	1	TO –	–	$(\Box \Diamond(x31) \wedge \Box \Diamond(x33)) \rightarrow (\Box \Diamond(x70))$
gb_woaf_30	76	4 ✓	8	398 ✓	1	TO –	–	$(x33 \wedge \Box \Diamond(x15)) \rightarrow (x57 \wedge \Box \Diamond(x58))$
gb_fair_40	97	82 ✓	22	TO –	–	TO –	–	$(\Box \Diamond(\neg x0)) \rightarrow (\Box \Diamond(x86 \wedge x86) \wedge \Box \Diamond(\neg x91))$
gb_trans_40	97	0.4 ✓	8	TO –	–	TO –	–	$(\Box \Diamond(x43)) \rightarrow (\Box \Diamond(x88))$
gb_woaf_40	97	62 ✓	14	TO –	–	TO –	–	$(\Box(\neg x1) \wedge \Box \Diamond(\neg x37)) \rightarrow (\Box(\neg x62) \wedge \Box \Diamond(\neg x82))$